ボットがマネージャーでないとこのイベントは流れません。ボットアカウントを**固定マネージャー**(推奨。ボットアカウントが先に DJ をフォローしている必要があります)か、この配信のマネージャーに指定します。指定がないとエラーには なりません — ストリームは `200` で開き `chat` も届くのに、`presence` だけが 1 件も届きません。手順は 下 にあります。配信の途中で指定・解除しても、接続し直さずに数秒以内に反映されます。

愛聴温度のしきい値はボットが決めます。サーバーが「何度から常連か」を決めてはくれません。`null` のときは判定しないのが正解です。

退場は届きません。`type` は現在 `JOIN` のみです。「今誰が部屋にいるか」が必要なら リスナー一覧を取得 で取得します。

同じ人が出て入るとまた届きます。上のコードのように `id` で重複を除きます。

`like` — ハート

リスナーがハートを送ると受け取ります。**必要な権限**: `events.like`

```
{
  "user": { "id": "Pq8Rs9Tu0Vw1Xy2Za3Bc4D", "nickname": "ラジオファン" },
  "type": "PAID",
  "totalAmount": 30,
  "amount": 10,
  "extraAmount": 5,
  "combo": 2,
  "time": "2026-09-03T02:59:03.249Z"
}
```

フィールド	型	null	説明
`user.id`	string	いいえ	送った人です。暗号化した値です。
`user.nickname`	string	**はい**	ニックネームです。
`type`	string	いいえ	お金がかかったかを示します。`PAID` はスプーンを使ったアイテムハート、`FREE` はそれ以外です。
`totalAmount`	number	いいえ	**今回配信に加算されたハートの総数** です。`(amount + extraAmount) × combo` です。

フィールド	型	null	説明
`amount`	number	いいえ	アイテム 1 つが与えるハート数です。
`extraAmount`	number	いいえ	ブーストアイテムで加算されたハート数です。なければ `0` です。
`combo`	number	**はい**	重ねて使った回数です。**無料ハートは `null` です。**
`time`	string	いいえ	送信時刻です。ISO-8601 UTC(`Z`)です。

`FREE` は 2 種類です — リスナーがタップで送る無料ハートと、ハートが 0 個で残高も減らない アイテムです。タップで送る無料ハートは `combo` が `null` です。

`FREE` もハートは加算されます。 `type` は反応の強さを分ける値であって、集計から外す値ではありません。

```
// stream.ts — handlers に `like: onLike` として入れます
let hearts = 0;

const onLike = (d: LikeEvent) => {
  hearts += d.totalAmount;                    // 集計は totalAmount — amount だけだとコンボとブーストが抜ける
  if (d.type === 'PAID') {
    console.log(`${d.user.nickname ?? '匿名'}さん、ハート ${d.totalAmount}個ありがとうございます！`);
  }
  // FREE は反応せず数えるだけ
};
```

`FREE` にいちいち反応すると配信が騒がしくなります。 タップで送るハートは降り続け、`PAID` はリスナーがアイテムを使ったもので重みが違います。通常は `PAID` にだけ 反応し、`FREE` は数えるだけにします。`FREE` にも反応するならクールダウンを置いてください。

集計には `totalAmount` を使います。 `amount` だけを足すとコンボとブーストが抜け、実際より 少なく数えます。`totalAmount` は配信サーバーが配信のハート数に加える値と同じです。

`like.amount` はハート数で `donation.amount` はスプーンです。 同じ名前で 単位が違い、計算式も違います。ハートは `(amount + extraAmount) × combo` である `totalAmount` を使い、ギフトの `amount` はす

でに合計なので再度掛けてはいけません。

`combo` は無料ハートでは `null`。です。そのまま計算に入れると落ちます。

`donation` — ギフト

リスナーがギフトを送ると受け取ります。**必要な権限**: `events.donation`

```
{
  "user": { "id": "Ab1Cd2Ef3Gh4Ij5Kl6Mn7O", "nickname": "スプーンリスナー" },
  "amount": 300,
  "message": "応援しています",
  "time": "2026-09-02T05:47:50.512Z"
}
```

フィールド	型	null	説明
`user.id`	string	いいえ	送った人です。暗号化した値です。
`user.nickname`	string	**はい**	ニックネームです。
`amount`	number	いいえ	今回のギフトの **合計スプーン** です。すでに掛け算済みです。
`message`	string	**はい**	一緒に届いた一言です。ない場合の方が多いです。
`time`	string	いいえ	送信時刻です。ISO-8601 UTC(`Z`)です。

```
// stream.ts — handlers に `donation: onDonation` として入れます
const spoons = new Map();

const onDonation = (d: DonationEvent) => {
  const total = (spoons.get(d.user.id) ?? 0) + d.amount;
  spoons.set(d.user.id, total);
  const note = d.message ?? '';          // null が既定なので ?? で受ける
  console.log(`${d.user.nickname ?? '匿名'}さん、${d.amount} スプーンありがとうございます！ ${note}`);
};
```

`amount` **はすでに掛け算済みの合計スプーンです。**単価ではないので、個数を再度掛けないで ください。
`like.amount` はスプーンではなくハート数です — 同じ名前でも単位が違います。

`message` は `null` の場合の方が多いです。空文字列ではないので `d.message.trim()` は落ちます。

配信全体の累計ギフトランキングはここから自分で積み上げます。ファンランキングを取得 は **今部屋にいる人** が対象なので、ギフトを送って 退出した人は外れます。

`end` — ストリーム終了

ストリームが **なぜ** 終わったかを知らせて閉じます。このフレームの後には何も届きません。**権限は不要です** — スコープが狭いボットも、接続が終わった理由は知る必要があります。

```
{ "reason": "LIVE_ENDED" }
```

フィールド	型	null	説明
`reason`	string	いいえ	`LIVE_ENDED`・`TOKEN_EXPIRED`・`RECONNECT`・`DUPLICATE_CONNECTION` のいずれかです。

`reason`	何が起きたか	どうするか
`LIVE_ENDED`	配信が終了しました。	再接続しません。次の配信を待ってから接続します。
`TOKEN_EXPIRED`	access token が失効しました。	トークンを更新して **すぐに** 接続し直します。
`RECONNECT`	サーバーがこの接続だけを閉じました。配信は続いています。	**バックオフなしですぐに** 接続し直します。トークンはそのまま使います。
`DUPLICATE_CONNECTION`	同じアプリが同じ DJ に新しく接続したため、この接続を閉じました。	**接続し直しません。** 新しい接続が引き継いでいます。

`RECONNECT` は **配信もトークンも問題ありません。** 更新するものはなく、バックオフを置いた 分だけイベントを取りこぼします。

このフレームが届かないのは **ネットワーク** で切れたときだけです。そのときはバックオフを 置いて接続し直します。

```
// stream.ts — handlers に `end: onEnd` として入れます。
```

```
// 理由ごとに対応が違います – ここでは次に何をするかだけを決め、再接続ループは外で回します
let next = null;

const onEnd = (d: StreamEndEvent) => {
  if (d.reason === 'LIVE_ENDED') next = 'wait';              // 配信終了。バックオフしてから接続
  else if (d.reason === 'TOKEN_EXPIRED') next = 'refresh';  // トークンを更新してすぐ接続
  else if (d.reason === 'DUPLICATE_CONNECTION') next = 'stop'; // 別の接続が引き継ぎました。接続し直しません
  else next = 'now';                                         // RECONNECT – バックオフなしで接続
  console.log(`ストリーム終了: ${d.reason} → ${next}`);
};
```

ボットをマネージャーに指定する方法

マネージャー指定は DJ が Spoon アプリで行います。ボットが API で要求することはできません。

運用するボットは固定マネージャーに指定することをおすすめします。 DJ がボットアカウントを固定マネージャーに指定すれば、配信のたびに指定し直す必要がありません。

- **ボットアカウントが先に DJ をフォローしている必要があります。** 配信中に DJ をフォロー(`POST /v1/live/follow`)を一度呼べば済みます。ボットアカウントで Spoon アプリにログインして直接フォローしてもかまいません。
- **固定マネージャーは DJ が配信中は変更できません。** フォロー API は逆に配信中にしか使えないので、配信中にフォローしておき、配信が終わってから DJ が指定すれば次の配信から適用されます。
- この配信の途中でマネージャーに指定・解除すると、数秒以内に反映されます。ボットが接続し直す必要はありません。

この配信だけ指定するときは、次の手順で行います。

1. **まずボットを配信に入れます。** 配信中の DJ に対してボットがイベントストリームへ接続すると、ボットアカウントがリスナーとして入場します。部屋にいる人だけがマネージャーに指定できます。
2. DJ が配信画面で **リスナー一覧を開き、ボットアカウントを選びます。**
3. プロフィールで **[マネージャー指定]** を押します。ひとつの配信でマネージャーは 3 名まで指定できます。
4. **数秒以内に反映されます。** 接続し直さなくても、その接続で `presence` が届き始めます。マネージャーを解除すると同じ時間内に止まります。

指定が必要なのは `presence` イベントです。

届かないもの

このストリームは **ドキュメントに記載されたイベントだけ** を送出します。配信サーバーに 新しいイベントが増えても、自動的に流れ出ることはありません。

個人メッセージ(DM)	特定のリスナーにだけ届くメッセージです。決して届きません。
マネージャー等級以上のメッセージ	ボットがマネージャーでなければ届きません。Spoon アプリと同じルールです。
ボット自身の発言・入場	自分が送ったものは自分に戻りません。エコーループを防ぐためです。同じ配信の **他のボットは見えます。**
システムイベント	配信メタの更新、スタジオ通知などは届きません。

受け取れる範囲はボットの部屋での権限に従います。 一般メンバーとして入場していれば リスナーがアプリ画面で見るものまで、DJ がマネージャーに指定していればマネージャー等級の メッセージまで届きます。個人メッセージ(DM)はどちらの場合も届きません。

接続ループ

配信が始まったかを別途確認する必要はありません。**配信中でなければ、ストリームのエンドポイントが 接続前に `404` を返します。** ボットのループはこれひとつで足ります。

```
ストリームに接続を試みる
  ├ 404 (OAPI_MNGR_0301) → 配信中ではありません。バックオフして再試行
  └ 200 → イベントを受信 … end{LIVE_ENDED}  → バックオフしてから接続の試行に戻る
                           end{RECONNECT} → バックオフなしですぐ接続の試行に戻る
                           end{DUPLICATE_CONNECTION} → 停止。別の接続が引き継いでいます
```

`GET /v1/live` で先に確認してから接続すると往復が 2 回になり、確認と接続の間に配信が終われば どのみち `404` を受け取ります。`live.read` なしで `events.chat` だけ同意されたボットでも、この ループはそのまま動きます。

`404` のたびに即座に再試行しないでください。 数秒の間隔を置きます — 再試行のたびにトークン 検証が走りますが、このエンドポイントには呼び出し上限がかかっていません。

`GET /v1/live` は配信の **情報** が必要なときに呼びます — タイトル・カテゴリ・リスナー数・`isChatFrozen` などです。

再接続の方針

ストリームは長く生きますが永遠ではありません。デプロイで Pod が再起動すると、開いていた 接続はすべて閉じます。再接続は選択ではなく必須です。

```
end(LIVE_ENDED)     → 配信が終了しました。バックオフして接続を試み直します
end(TOKEN_EXPIRED) → トークンを更新してすぐに接続し直します
end(RECONNECT)      → サーバーが接続だけを閉じました。バックオフなしですぐに接続し直します
end(DUPLICATE_CONNECTION) → 同じアプリの別の接続が引き継ぎました。接続し直しません
理由なく切断          → 指数バックオフ(1 秒 → 2 秒 → 4 秒 → … 最大 30 秒)+ jitter で再接続します
404                 → 配信中ではありません。エラーではなく状態なのでバックオフして接続し直します
401                 → 再連携が必要です。人の介入が要るので無限リトライはしません
403(detailCode なし) → 権限が足りません。再同意だけで済み、連携は生きています
403 OAPI_MNGR_0209  → DJ がボットをブロックしました。再試行では解決しないので止めて DJ に案内します
```

`401` と `403` は別々に処理します。 `401` は DJ が連携し直さないと解消せず、権限不足の `403` は権限を追加で得るだけで済みます。まとめて扱うと、問題のない連携まで切ってしまいます。また `401` を無限にリトライすると、永遠に失敗しながら呼び出し上限だけを消費します。

常駐ボットが 1 時間に 1 回 `TOKEN_EXPIRED` を受け取るのは正常です。 ストリームは接続に使った access token の 失効時刻(発行から 1 時間)に終わります。事前に更新しておいても、すでに開いているストリームはその時刻に終わるので、`TOKEN_EXPIRED` を受け取ったら refresh token で更新してすぐに接続し直します。この再接続でも入場メッセージは再び表示されません。

再接続しても、切れていた間のチャットは受け取れません。`Last-Event-ID` による再開はまだ 開いていません。

よくある間違い

`:` で始まる行をパースしようとしないでください。 ハートビートです。`data:` がないフレームは 読み飛ばします。

時刻の小数部の桁数を固定しないでください。 ISO-8601 のパーサーを使い、`%Y-%m-%dT%H:%M:%S.%fZ` のような書式文字列は避けます。

圧縮を期待しないでください。 `Accept-Encoding: gzip` を送ってもこのエンドポイントは 圧縮しません。バッファにたまってから出るとリアルタイムでなくなるからです。

ストリームが終わっても配信が終わったとは限りません。 `end` フレームの `reason` を見てください。

入場メッセージが繰り返されるのは再接続のせいではありません。 再接続だけでは入場メッセージは再び表示されません。ボットが部屋から抜けて入り直したということなので、次を確認します。

- DJ がボットを強制退出させていないか
- 最初に入場してから 5 時間が経ち、入場記録が失効していないか

フィールドごとの落とし穴(`nickname` の `null`、ハート集計に `totalAmount` を使うこと)は リアルタイムイベントの権限 の各節にあります。

エラー

HTTP	`detailCode`	何が起きたか	どうするか
`401`	なし	トークンがないか無効です。失効・破棄・DJ の連携解除・アプリ停止がすべてここに入ります。	DJ に再連携を案内します。
`403`	なし	`events.` で始まる権限がひとつもありません。	権限を追加して再同意を得ます。
`403`	`OAPI_MNGR_0209`	配信に入場できません。DJ がボットをブロックした可能性があります。	再試行では解決しません。DJ に案内します。
`404`	`OAPI_MNGR_0301`	配信中ではありません。	バックオフして接続し直します。
`502`	`OAPI_MNGR_2302`	配信サーバーのエラーです。	バックオフしてから再試行します。

ストリームが開く前のエラーは SSE フレームで届きます。 このエンドポイントは `text/event-stream` で応答するため、接続に失敗してもボディが `data:` で始まります。ボディをそのまま JSON としてパースすると落ちます。**HTTP ステータスコードで判別し**、ボディが必要なら `data:` を取り除いてからパースします。

この表にない状態も返ることがあります。 誤ったメソッド・`Content-Type`(`400`・`405`・`415`、`OAPI_MNGR_0103`)・呼び出し上限(`429`、ボディなし)・サーバーエラー(`500`、`OAPI_MNGR_1002`)は **すべてのエンドポイントに共通** です — エラーコード にあります。

関連ドキュメント

- エンドポイント一覧 — エンドポイントの全一覧です。
- 現在の配信を取得 — タイトル・リスナー数・`isChatFrozen` です。
- リスナー一覧を取得 — 同じ `id` で部屋にいる人を確認します。
- チャットを送る — 受け取ったイベントに反応して発言します。
- エラーコード — エラーコードの一覧です。